

Day 7 - Mini Project: CLI Task Tracker (Week 1 Review)

Day 7 — Mini Project: CLI Task Tracker (Week 1 Review)

Objectives

Combine everything from Days 1-6 into one working program:

- variables/types/mutability (Day 1)
- control flow, loops (Day 2)
- lists and dicts as your data model (Day 3)
- functions with clear parameters and return values (Day 4)
- string formatting and file I/O (Day 5)
- exception handling and modules (Day 6)

This is intentionally less hand-held than the previous days — the point of a review project is to make the design decisions yourself, using what you've learned, and get stuck in the productive way. That said, this lesson explains every new piece of vocabulary you'll need (command-line arguments, JSON) in full before you start, exactly like every other day — "less hand-held" means fewer step-by-step instructions on *how* to build it, not less explanation of the *new* concepts involved.

What's actually new today, explained fully

Two things show up in this project that you haven't formally used yet: **command-line arguments** and **JSON**. Both are explained below before you start.

Command-line arguments — how a program receives instructions typed after its name

So far, every script you've run has been `python somefile.py`, with nothing after the filename. But you've also been typing things like `python exercises.py` with no extra words — today, you'll write a program that pays attention to *extra words typed after the filename*, called **command-line arguments**. For example: `python tasks.py add "Buy milk"` — here, `add` and `"Buy milk"` are two arguments your program can read and react to differently.

Python gives you access to these through a list called `sys.argv`, provided by the built-in `sys` module (recall from Day 6: a "module" is a file of pre-written Python code you gain access to via `import`):

```
import sys
print(sys.argv)
```

If you ran this as `python tasks.py add "Buy milk"`, `sys.argv` would be the list `["tasks.py", "add", "Buy milk"]`. Notice `sys.argv[0]` is *always* the name of the script itself — the arguments you actually care about start at index 1. So `sys.argv[1]` would be `"add"`, and `sys.argv[2]` would be `"Buy milk"`. This is just an ordinary Python list — everything you learned about lists on Day 3 (indexing, slicing, `len()`) applies to it

directly.

This project is your first time using `sys.argv` for real, which is why the instructions below explicitly walk you through reading commands out of it by hand, rather than reaching for a fancier tool (`argparse`, which you'll meet in later, more advanced work) — doing it manually once, here, builds real intuition for what command-line arguments actually are underneath any tool that makes it more convenient later.

JSON — saving Python data to a file so it survives between runs

Every program you've written so far forgets everything the instant it finishes running — all your variables simply cease to exist. Today's project needs to **persist** data — meaning: save it somewhere so that the *next* time you run the program (even tomorrow, even after restarting your computer), it remembers what was saved before.

JSON (JavaScript Object Notation — the name comes from another programming language, but it's a completely universal, language-independent text format at this point) is a simple, human-readable way of writing down data — lists, dicts, strings, numbers, True/False, None — as plain text, so it can be saved to a file and read back later, by Python or by any other programming language. Python's built-in `json` module converts back and forth between "real" Python data (in memory, while your program is running) and JSON text (saved permanently on disk):

```
import json

data = [{"description": "Buy milk", "done": False}]

with open("tasks.json", "w") as f:
    json.dump(data, f)          # takes a Python object (here, a list of dicts) and WRITES it as JSON

with open("tasks.json") as f:
    loaded = json.load(f)       # READS the JSON text back out of the file and turns it back into Python
```

Notice the naming pattern: `dump` = take a Python value and write it out as JSON (think: "dump this data out to a file"). `load` = read JSON text back in and reconstruct the Python value it represents. You'll get the full, deeper treatment of JSON (including reading it from web APIs) on Day 11 — for today, this much is genuinely everything you need.

The brief

Build a **command-line task tracker** called `tasks.py` that a user runs repeatedly from the terminal to manage a to-do list that persists between runs (saved to a JSON file, see the note below on `json` — you'll cover it properly Day 11, but `json.dump/json.load` are enough to use today).

Required features

- 1 `python tasks.py add "Buy milk"` — adds a new task, not yet complete
- 2 `python tasks.py list` — prints all tasks with an index number and a `[ ]/[x]` complete marker
- 3 `python tasks.py done 2` — marks task #2 as complete
- 4 `python tasks.py remove 2` — deletes task #2
- 5 Tasks persist in a file (e.g. `tasks.json`) between runs — running `list` after closing and reopening your terminal should still show your tasks

Suggested data model

A list of dicts, e.g.:

```
[
    {"description": "Buy milk", "done": False},
    {"description": "Walk dog", "done": True},
]
```

Suggested architecture (you decide the details)

- A function to load tasks from the JSON file (handle the file-not-existing-yet case — first run — without crashing: this is exactly the exception-handling judgment call from Day 6)
- A function to save tasks back to the JSON file
- One function per command (`add_task`, `list_tasks`, `complete_task`, `remove_task`)
- Use `sys.argv` to read command-line arguments (`import sys`; `sys.argv[0]` is the script name, `sys.argv[1]` is the first real argument, etc.) — you haven't been taught `argparse` yet on purpose; parsing `sys.argv` by hand is a good forcing function to practice list indexing, slicing, and `try/except` around `int(sys.argv[2])`

Minimal JSON usage you need (full treatment on Day 11)

```
import json

with open("tasks.json", "w") as f:
    json.dump(data, f)          # write a Python object as JSON text

with open("tasks.json") as f:
    data = json.load(f)         # read JSON text back into a Python object
```

Edge cases worth handling (this is where the real learning is)

- What happens if the user runs `list` before ever running `add`? (file doesn't exist yet)
- What happens if the user runs `done 99` but there are only 3 tasks? (index out of range)
- What happens if the user runs `done abc` (not a number)?
- What happens if the user runs the script with no arguments, or an unrecognized command?

None of these should crash with an ugly traceback — they should print a clear, human-readable message. This is the difference between "code that works on the happy path" and "code you could actually ship."

How to approach this (process, not just code)

- 1 Sketch the file structure and function signatures on paper/in comments FIRST, before writing logic — this is what real engineers do before touching a keyboard for anything non-trivial.
- 2 Build `add` and `list` first, test manually by running the script from the terminal, before adding `done/remove`.
- 3 Get the happy path fully working before handling edge cases.
- 4 Once it all works, go back and deliberately try to break it (the edge cases above) and fix what breaks.

A note on scope

Don't reach for classes, `argparse`, or anything from Week 2 — the whole point is proving you can build something real with *only* Week 1 material. You'll rebuild something similar with classes and better tooling by Day 14.

What's provided

- `starter.py` — an empty scaffold with function signatures and docstrings to get you started (optional to use — you can also start from a blank file)
- `solution.py` — a full reference implementation. **Build your own first.** This project has many equally valid designs; comparing your approach to this one afterward is valuable, but only after you've made your own decisions and hit your own bugs.

Manual test checklist

Run these in order from your terminal in this folder and confirm the output matches expectations:

```
python tasks.py add "Buy milk"
python tasks.py add "Walk dog"
python tasks.py list
python tasks.py done 1
python tasks.py list
python tasks.py remove 2
python tasks.py list
python tasks.py done 99
python tasks.py done abc
python tasks.py bogus-command
python tasks.py
```